

# Feature-Preserving Graph Coarsening for Node Classification using GATs

Supervisor: Prof. Preety Singh

**Nityansh Pant**

Department of Mathematical Sciences  
Rajiv Gandhi Institute of Petroleum Technology

Summer Research Internship

July 27, 2026

# Motivation and Research Question

## Graph learning context

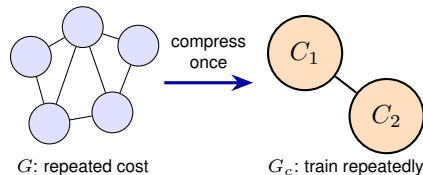
GNNs combine node attributes with graph context. This is valuable in citation networks because both paper content and citation relations carry class information. GATs further learn which neighbours deserve greater weight, allowing less informative messages to be suppressed.

For  $L$  layers,  $H$  heads, hidden width  $d_h$ , and  $|E|$  stored edges,

$$\text{GAT edge cost} \approx \mathcal{O}(LH|E|d_h).$$

## Research question

Can we reduce  $|V|$  and  $|E|$  before GAT training while preserving the features and structure needed for node classification?



**Figure:** A one-time coarsening step reduces the graph processed at every training epoch.

# Existing Methods and Research Gap

| Method              | Primary signal            | Advantage                       | Limitation                                                                     |
|---------------------|---------------------------|---------------------------------|--------------------------------------------------------------------------------|
| Heavy Edge Matching | Edge weights              | Fast local contraction          | May merge structurally strong but feature-dissimilar nodes                     |
| Graclus             | Normalized graph cuts     | Efficient multilevel clustering | Primarily structure-driven; node attributes are not central                    |
| DiffPool            | Learned dense assignments | Task-adaptive clusters          | Additional parameters and dense assignments raise memory and optimization cost |

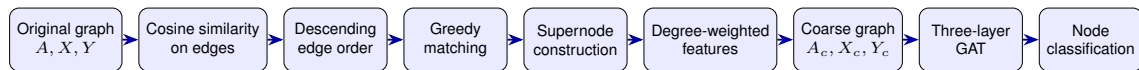
**Table:** Trade-offs among representative graph-coarsening approaches.

## Research gap

Existing methods either ignore feature similarity, produce dense assignments, or require expensive optimization. We therefore seek a sparse, local, feature-aware rule computed before GAT training.

Sources: Dhillon, Guan, and Kulis (2007); Ying et al. (2018).

# Proposed Pipeline



**Figure:** The proposed static coarsening stage followed by GAT learning on the compressed graph.

## Local pairing

Connected, feature-similar nodes are merged into singleton or pair clusters.

## Feature preservation

Topology restricts valid merges, while feature affinity identifies the safest local pairing.

## Efficient learning

The resulting sparse graph reduces repeated attention computation without discarding nodes.

# Feature-Preserving Single-Pass Greedy Matching

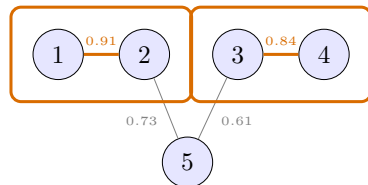
## Affinity and assignment

For each existing edge  $(u, v) \in E$ ,

$$s(u, v) = \frac{x_u^\top x_v}{\|x_u\|_2 \|x_v\|_2}, \quad M_{ir} = \mathbb{1}[i \in C_r].$$

Cosine similarity is preferred to Euclidean distance because sparse bag-of-words vectors are compared by direction, not magnitude.

- 1 Compute  $s(u, v)$  once for all stored edges.
- 2 Sort edges in descending similarity.
- 3 Match an edge only if both endpoints are unmatched.
- 4 Permanently mark the pair and skip conflicting incident edges.
- 5 Preserve every remaining node as a singleton cluster.



$$C_1 = \{1, 2\}, \quad C_2 = \{3, 4\}, \quad C_3 = \{5\}$$

**Figure:** High-affinity disjoint pairs are selected; unmatched nodes remain represented.

## Single-pass matching

Similarities remain fixed after their initial computation. Recomputing them after every merge would turn the method into hierarchical agglomerative clustering; the fixed ordering keeps preprocessing near  $\mathcal{O}(|E| \log |E|)$  while retaining highly similar local pairs.

# Coarse Graph Construction

## Degree-weighted features

$$x_r^c = \frac{\sum_{i \in C_r} (d_i + 1) x_i}{\sum_{i \in C_r} (d_i + 1)}.$$

Higher-degree members carry more structural context than a simple average reflects, while +1 keeps every node influential. The resulting representative combines semantic similarity with structural importance.

## Coarse adjacency

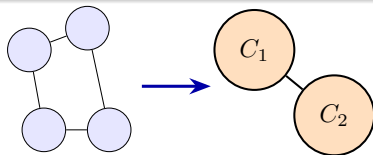
$$\tilde{A}_c = M^\top A M, \quad (A_c)_{rs} = \mathbb{I}[(\tilde{A}_c)_{rs} > 0], \quad r \neq s.$$

Every inter-cluster edge induces a supernode connection. Duplicate edges and self-loops are removed to obtain a simple sparse graph.

## Majority-vote labels

$$y_r^c = \text{mode}\{y_i : i \in C_r\}.$$

A supernode requires one target label, so the mode represents its dominant class without introducing an additional learnable label model. This provides deterministic and inexpensive coarse supervision.



**Figure:** Cluster assignments project original nodes, edges, features, and labels to the coarse graph.

# Training Dataset Construction: Pure-Cluster Masks

## How masks are created

A cluster is eligible when all members

- 1 share the same class label, and
- 2 originate from one original data role.

Eligible clusters are shuffled with a fixed seed and redistributed into 80/10/10 train, validation, and test masks.

## Recorded masks

|            |     |
|------------|-----|
| Training   | 752 |
| Validation | 94  |
| Test       | 93  |

## Label-consistent supervision

Mixed-label supernodes create label ambiguity. Assigning one target would provide contradictory supervision and destabilize GAT training. Only pure clusters enter the supervised loss, preserving label consistency.

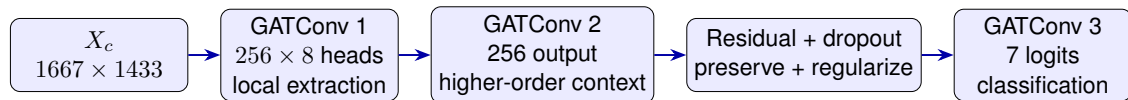
## Important distinction

Mixed clusters remain in the coarse graph and still contribute features and messages during GAT propagation; they are excluded only from supervised loss and accuracy masks.

## Evaluation scope

The redistributed coarse split is not directly comparable with the standard Cora split.

# Three-Layer GAT Architecture



**Figure:** The implemented GAT transforms coarse features into seven class logits.

## Layer 1

Multiple heads extract complementary local patterns.

## Layer 2

A second propagation step captures broader coarse neighbourhoods.

## Residual

The skip connection preserves low-level information and supports stable gradient flow.

## Layer 3

The final transformation maps the learned representation to the seven Cora classes.



# Training Objective and Optimization

## Cross-entropy classification loss

$$\mathcal{L}_{\text{cls}} = \text{CE}(Z_{\text{train}}, Y_{\text{train}}^c)$$

with label smoothing 0.15.

This term trains the class logits, while label smoothing reduces overconfidence and stabilizes supervision on the smaller coarse dataset.

## Feature compactness diagnostic

$$\mathcal{L}_{\text{feature}} = \frac{1}{n} \|X - MX_c\|_F^2, \quad \mathcal{L} = \mathcal{L}_{\text{cls}} + 0.01\mathcal{L}_{\text{feature}}.$$

This quantity measures how well supernode features reconstruct their member features and therefore provides a direct feature-preservation diagnostic.

## AdamW

AdamW decouples weight decay from the adaptive gradient update, providing clearer regularization of the GAT weights than applying an  $L_2$  term through Adam's update.

## Cosine annealing

$\eta_0 = 10^{-3}$ ,  $T_{\text{max}} = 400, 500$  epochs.

The cosine schedule gradually reduces the learning rate after rapid early progress, enabling finer late-stage optimization without manually selected step changes.

# Experimental Results

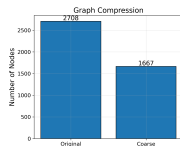


Figure: Original and coarse node counts.

| Metric         | Result  |
|----------------|---------|
| Coarse nodes   | 1667    |
| Node reduction | 38.44%  |
| Edge reduction | 33.82%  |
| Runtime        | 55 s    |
| Components     | 78 → 78 |

Table: Compression, runtime, and connectivity.

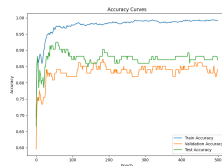


Figure: Training, validation, and test accuracy across 500 epochs.

## Predictive performance

Best validation: 0.8723

Best test: 0.9247

## Interpretation boundary

Accuracy uses redistributed pure-cluster masks rather than the standard Cora split; simulated memory graphics are not treated as measured evidence.

# Conclusion and Future Work

## Contributions

- ✓ Feature-aware graph coarsening
- ✓ Static cosine-based greedy matching
- ✓ Degree-weighted feature aggregation
- ✓ Pure-cluster supervised learning
- ✓ Efficient GAT training on the coarse graph

## Takeaway

The pipeline reduces graph size while retaining strong accuracy on the implemented coarse split, using a transparent and inexpensive preprocessing rule.

## Future work

- Differentiable assignment matrices  $M_\phi$
- End-to-end trainable coarsening
- Original-node prediction transfer
- Larger benchmark datasets
- Measured memory and runtime baselines

# References

- [1] P. Sen, G. Namata, M. Bilgic, L. Getoor, B. Gallagher, and T. Eliassi-Rad.  
“Collective Classification in Network Data.”  
*AI Magazine*, 29(3), 2008.
- [2] I. S. Dhillon, Y. Guan, and B. Kulis.  
“Weighted Graph Cuts without Eigenvectors: A Multilevel Approach.”  
*IEEE Transactions on Pattern Analysis and Machine Intelligence*, 29(11), 2007.
- [3] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio.  
“Graph Attention Networks.”  
*International Conference on Learning Representations*, 2018.
- [4] R. Ying, J. You, C. Morris, X. Ren, W. Hamilton, and J. Leskovec.  
“Hierarchical Graph Representation Learning with Differentiable Pooling.”  
*Advances in Neural Information Processing Systems*, 2018.
- [5] A. Loukas.  
“Graph Reduction with Spectral and Cut Guarantees.”  
*Journal of Machine Learning Research*, 20(116), 2019.
- [6] I. Loshchilov and F. Hutter.  
“Decoupled Weight Decay Regularization.”  
*International Conference on Learning Representations*, 2019.

# **Thank You**

Questions?

Nityansh Pant